OII462 · Tarea 6

Encoder estático, complejidad y variantes del Transformer para el WTAP

Informe de los puntos 1, 2, 3, 5 y 6

[Nombre(s) del grupo] · Optimización combinatoria · 2026

Resumen. En el módulo 2, nuestro modelo volvía a procesar toda la instancia en cada decisión, porque 12 de sus 16 datos de entrada cambiaban después de cada asignación. En esta tarea lo rediseñamos en dos partes con responsabilidades separadas: un encoder que lee una sola vez los datos fijos de la instancia (el valor de cada objetivo y las probabilidades de impacto) y un decoder que, en cada paso, mira el estado actual y elige el siguiente par arma → objetivo. Así, el costo de cada decisión crece en proporción al tamaño de la instancia y no a su cuadrado. Probamos además dos variantes: A, que le da al modelo una pista de la heurística clásica MMR, y B, que deja que los pares intercambien información en cada paso sin perder esa eficiencia. En las instancias de prueba de 10×8, los tres modelos quedan a 2,2–2,4 % del experto, contra 7,7 % de MMR y 3,0 % del modelo del módulo 2, y son parecidos entre sí.

Términos usados en este informe

Término	Significado
Instancia W×T	Un problema con W armas y T objetivos. Las acciones posibles son los n = W·T pares arma → objetivo.
Experto ILS	El algoritmo de búsqueda local de las tareas anteriores. El modelo lo imita y lo usamos como referencia (en las instancias pequeñas verificadas encuentra el óptimo).
Gap	Cuánto peor es una solución que la del experto, en porcentaje: 0 % significa igual de buena.
MMR	Heurística clásica que, en cada paso, asigna el par que más reduce el valor esperado que sobrevive.
Encoder y decoder	Las dos partes del modelo: el encoder lee la instancia y el decoder toma las decisiones una a una.
Época y semilla	Una época es una pasada por todos los datos de entrenamiento. Cada semilla es una repetición del entrenamiento con otro punto de partida aleatorio.

1. Rediseño de la arquitectura (punto 1)

El modelo del módulo 2 recibía una tabla con una fila por cada par arma → objetivo y 16 datos por fila. Al recorrer una construcción completa comprobamos que 12 de esos datos cambian después de cada asignación (por ejemplo, si el arma ya fue usada o cuánto valor queda en el objetivo). Por eso todo el modelo debía recalcularse en cada decisión: no tenía un encoder propiamente tal, y el costo de cada decisión crecía con el cuadrado del número de pares.

- **Encoder (una vez por instancia).** Recibe solo lo que no cambia: el valor V_j de cada objetivo y las probabilidades de impacto p_{ij}. Como las armas no tienen datos propios, todas parten del mismo vector aprendido (un token aprendible) y se diferencian por sus probabilidades, que el modelo usa como conexiones entre armas y objetivos: cada arma se informa de los objetivos que puede alcanzar, y cada objetivo, de las armas que lo pueden atacar.
- **Decoder (en cada decisión).** Recibe el estado: qué armas siguen libres y cuánto valor sobrevive en cada objetivo. Combina esa información con lo que entregó el encoder, da un puntaje a cada par arma libre → objetivo y elige el de mayor puntaje.

Además, el modelo ya no recibe pistas calculadas con MMR, como el del módulo 2: debe aprenderlas por su cuenta a partir de los valores, las probabilidades y el estado.

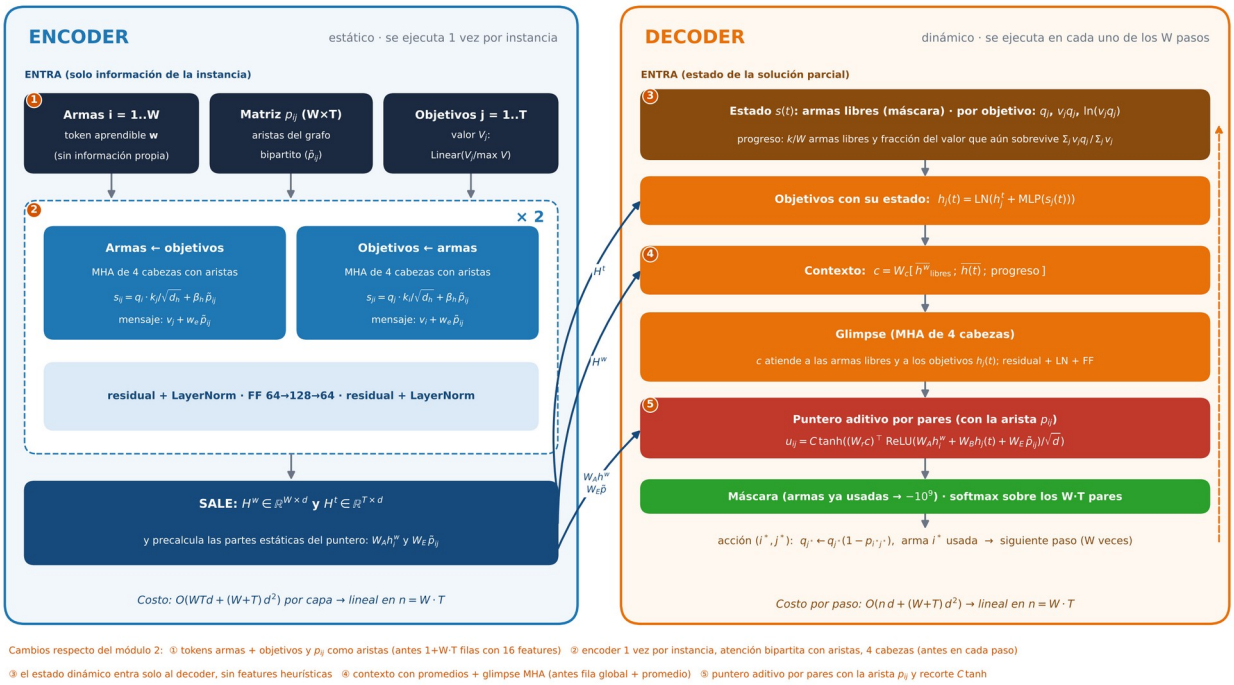


Figura 1. Arquitectura base rediseñada. Izquierda: el encoder, que se calcula una vez por instancia solo con los datos fijos. Derecha: el decoder, que se repite en cada decisión y usa el estado actual. Los círculos naranjos numerados marcan los cambios respecto del módulo 2 (ver la tabla).

La tabla resume los cambios, con la numeración de la Figura 1:

#	Qué cambió	Módulo 2	Ahora
1	Qué recibe el modelo	Una fila por par arma → objetivo, con 16 datos	Armas y objetivos por separado; las probabilidades, como conexiones entre ellos
2	Encoder	Se recalculaba en cada decisión	Se calcula una sola vez por instancia
3	Estado de la solución	Mezclado con los datos de entrada, junto con pistas de MMR	Solo en el decoder: armas libres y valor que sobrevive en cada objetivo
4	Resumen del estado	Una fila global y el promedio de las filas disponibles	Promedios de las armas libres y de los objetivos, refinados con atención
5	Puntaje de cada par	Comparaba cada fila con el resumen	Combina la información del arma, del objetivo y de su probabilidad de impacto
	Entrenamiento	60.000 decisiones sueltas	Instancias completas: una lectura del encoder y W decisiones

2. Complejidad temporal (punto 2)

Llamamos n = W·T al número de pares arma → objetivo. Un costo es **lineal** si crece en proporción a n (una instancia el doble de grande toma el doble de tiempo) y **cuadrático** si crece con n² (el doble de grande toma cuatro veces más). El decoder se ejecuta W veces por instancia, así que su costo es el que más importa; el enunciado pide que sea a lo más lineal.

Modelo	Encoder (una vez por instancia)	Decoder (en cada decisión)
Módulo 2	No existe por separado: todo se recalcula en cada paso	Cuadrático, O(n²): cada par se compara con todos los demás
Base y variante A	Lineal, O(n): cada arma mira los T objetivos y cada objetivo las W armas	Lineal, O(n): se revisa cada par una vez
Variante B (con ISAB)	Lineal, O(n)	Lineal, O(n·m): los pares se comunican a través de m = 16 resúmenes
Variante B sin ISAB (solo para comparar)	Lineal, O(n)	Cuadrático, O(n²)

Un decoder lineal es lo mejor posible, porque en cada decisión hay que dar un puntaje a cada uno de los n pares. El encoder también es lineal, aunque la tarea permitía que fuera cuadrático. La única parte que podría volverse cuadrática es la comunicación entre pares de la variante B, y por eso usa ISAB (sección 3.2).

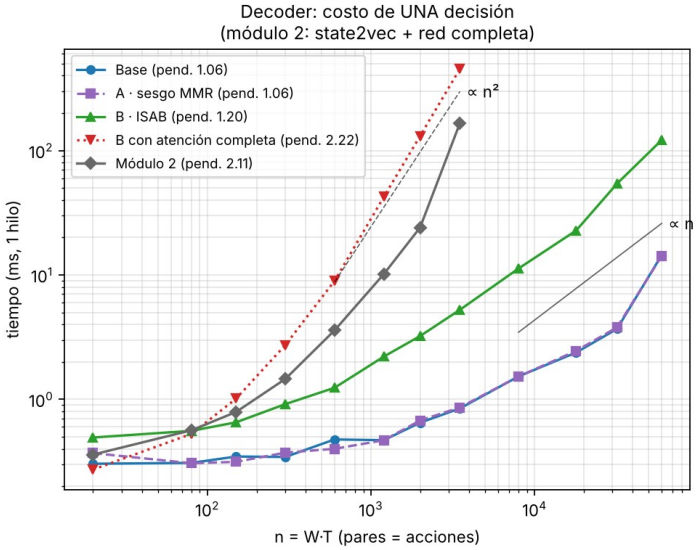


Figura 2. Tiempo de una decisión según el número de pares n, medido en un hilo de CPU (ambos ejes en escala logarítmica). Las curvas más empinadas, del módulo 2 y de B sin ISAB («B con atención completa» en la leyenda), crecen como n²; las de los modelos nuevos, como n.

Las mediciones confirman el análisis: en una instancia de 50×40, una decisión pasa de 24,0 ms en el módulo 2 a 0,65 ms en la base, y resolver la instancia completa es 36 veces más rápido. El encoder cuesta 0,48 ms en 10×8 y se paga una sola vez.

3. Variantes (punto 3)

3.1 Variante A: una pista de MMR (sesgo inductivo)

Un sesgo inductivo es una pista que le damos al modelo para que no parta de cero. Usamos la de MMR: cuánto valor esperado elimina asignar el arma i al objetivo j en el estado actual, $g_{ij} = V_j \cdot q_j \cdot p_{ij}$, donde q_j es la probabilidad de que el objetivo j siga en pie. La pista se suma en dos lugares, como en el material del curso:

- **En el puntaje de cada par (decoder):** los pares con mayor ganancia de MMR reciben más puntaje. Al comienzo el modelo elige exactamente lo mismo que MMR, y durante el entrenamiento aprende cuándo conviene apartarse.
- **En la atención del encoder:** cada arma presta más atención a los objetivos donde su ganancia inicial es mayor.

En ambos casos la pista se multiplica por pesos α que el modelo aprende, así que él decide cuánto confiar en ella.

¿Por qué MMR es un criterio robusto?

- Es la heurística clásica del WTAP (den Broeder et al., 1959) y es óptima cuando todas las armas son iguales.
- Es el punto de partida de métodos del estado del arte (Ahuja et al., 2007) y de nuestro propio experto.
- En nuestros datos, el experto elige la misma acción que MMR en el 80 % de las decisiones, y esa acción está entre las 3 mejores de MMR en el 99 % de los casos.

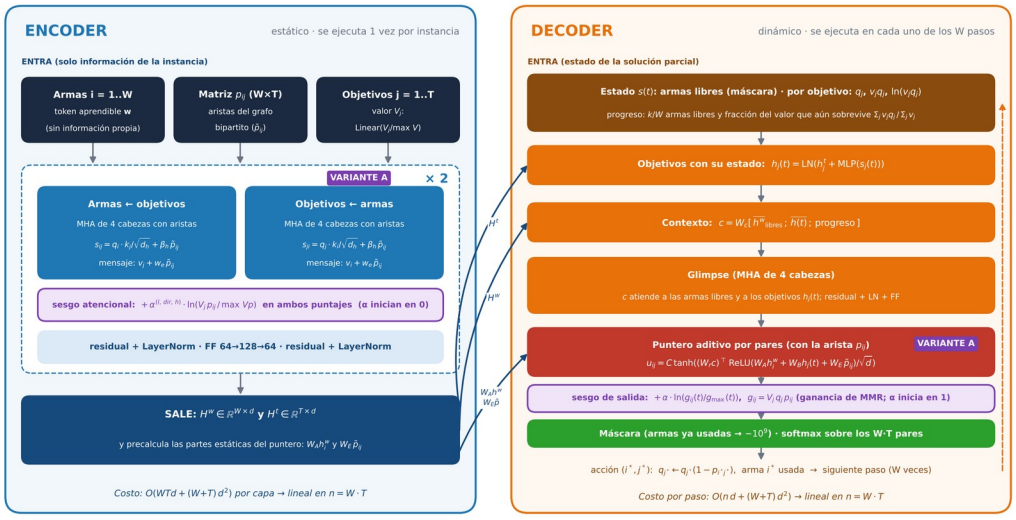


Figura 3. Variante A: la base más la pista de MMR (en morado), en el encoder y en el puntaje final.

3.2 Variante B: los pares se comunican (ISAB)

En la base, cada par se evalúa por separado: un par no "sabe" si otra arma libre aprovecharía mejor su objetivo. En la variante B, en cada decisión los pares intercambian información. Hacerlo comparando cada par con todos los demás sería cuadrático, así que usamos un bloque ISAB (Lee et al., 2019): 16 puntos aprendidos resumen el conjunto de pares y luego cada par lee esos resúmenes. Así la comunicación cuesta O(n·m) en vez de O(n²).

¿Por qué ISAB? Lo elegimos frente a LUNA (Ma et al., 2021) porque el estado es un conjunto sin orden que cambia en cada paso, sin una memoria que deba conservarse, que es lo que aporta LUNA. Tampoco usamos atención dispersa, porque necesitaría una regla para decidir qué pares conectar, y ese papel ya lo cumple la pista de MMR de la variante A. Así, cada variante prueba una sola idea.

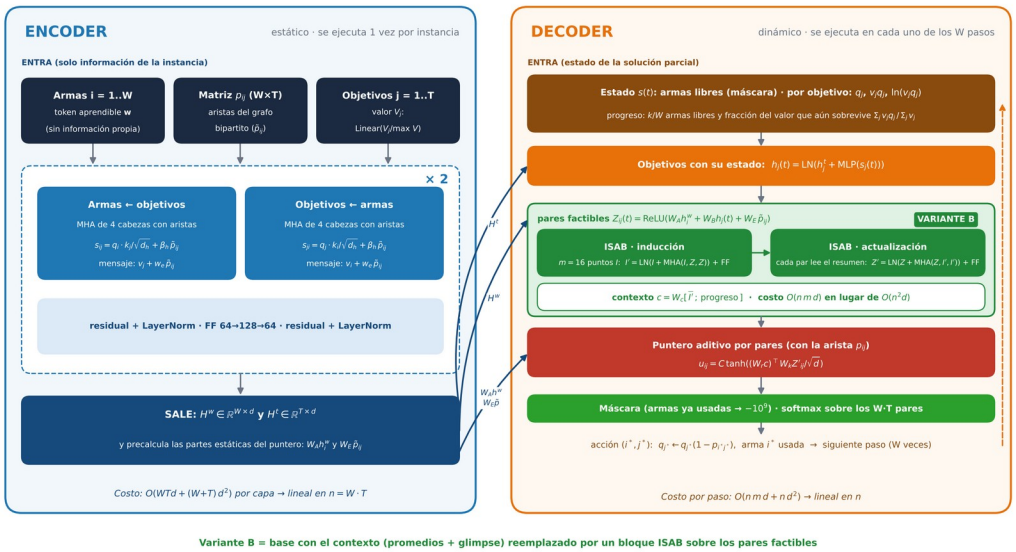


Figura 4. Variante B: el resumen del estado de la base se reemplaza por un bloque ISAB sobre los pares disponibles (en verde).

4. Entrenamiento (punto 4)

Los tres modelos se entrenaron exactamente igual: con los mismos datos (6.000 instancias de 10×8 resueltas por el experto y 300 de validación), el mismo optimizador y la misma tasa de aprendizaje, hasta 60 épocas con parada temprana (el entrenamiento se detiene si deja de mejorar) y 3 semillas por modelo. La lista completa de hiperparámetros está en el notebook.

5. Convergencia (punto 5)

La Figura 5 muestra las curvas de entrenamiento y la tabla resume la velocidad de cada modelo:

Modelo	Gap antes de entrenar	Épocas hasta gap < 3 %	Mejor pérdida de validación	Segundos por época
Base	821 %	10–13	0,317 ± 0,006	3,6
A · sesgo MMR	6,9 %	9–12	0,311 ± 0,007	3,7
B · ISAB	821 %	11–14	0,293 ± 0,005	15,1

Épocas: rango entre las 3 semillas. Pérdida: promedio ± desviación entre semillas (menor es mejor).

- **Velocidad, en épocas.** A parte con ventaja: como arranca imitando a MMR, su gap antes de entrenar es 6,9 %, contra 821 % de los otros dos (sin entrenar, mandan todas las armas al mismo objetivo). Pero la ventaja dura poco: después de la primera época los tres ya superan a MMR y llegan a un gap menor a 3 % en 9 a 14 épocas.
- **Velocidad, en tiempo real.** B es la que más rápido baja la pérdida y la que termina más abajo, pero cada época le toma 4,2 veces más (15,1 s contra 3,6–3,7 s): medida en minutos, es la más lenta.
- **Estabilidad.** Las 9 corridas (3 modelos × 3 semillas) convergieron sin saltos ni divergencias, y las semillas de un mismo modelo terminan muy cerca. Al final, los tres llegan a un gap de validación parecido (2,19–2,35 % en las últimas 10 épocas): la menor pérdida de B no se tradujo en mejores soluciones.
- **La pista de MMR se usa.** El peso α con que la variante A suma la pista de MMR subió de 1 a 2,0–3,4 durante el entrenamiento: el modelo aprendió a confiar más en ella.

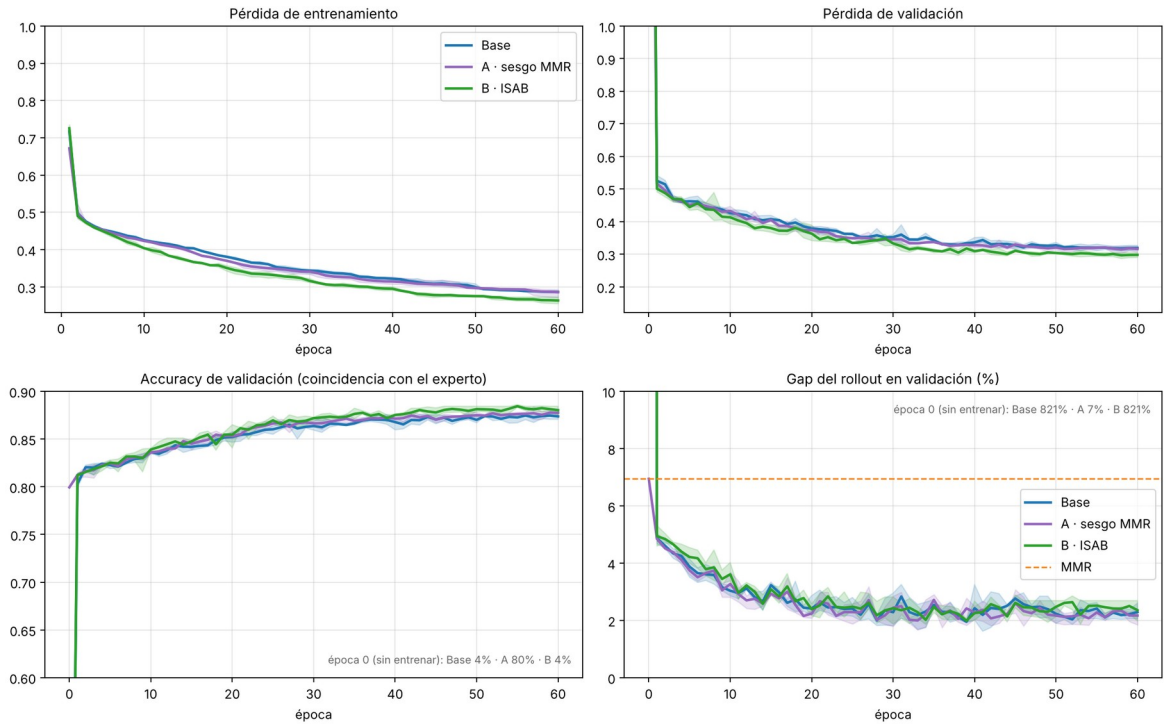


Figura 5. Curvas de entrenamiento por época (línea = promedio de 3 semillas; banda = mínimo y máximo). Abajo a la derecha, el gap de las soluciones que construye cada modelo en 100 instancias de validación; la línea naranja es MMR.

6. Evaluación con instancias de distintos tamaños (punto 6)

Los modelos se entrenaron solo con instancias de 10×8 y los probamos, sin cambios, en 7 tamaños, de 5×4 a 100×80. Usamos las mismas instancias que el proyecto del módulo 2 (30 por tamaño; las 50 de prueba en 10×8), así que la columna «Módulo 2» es directamente comparable. Cada valor es el gap respecto del experto: promedio ± desviación estándar sobre las 3 semillas y todas las instancias.

Tamaño	Base	A · sesgo MMR	B · ISAB	MMR	Módulo 2
5×4	**0,71 ± 2,61**	1,00 ± 3,36	1,30 ± 3,99	6,46 ± 7,87	1,67 ± 4,21
10×8	2,33 ± 2,93	2,44 ± 2,96	**2,21 ± 3,13**	7,68 ± 4,91	2,97 ± 3,68
15×10	**3,13 ± 2,57**	3,40 ± 2,74	3,25 ± 3,03	7,82 ± 4,92	5,28 ± 3,87
20×15	**1,96 ± 1,69**	2,09 ± 1,58	2,01 ± 1,86	5,71 ± 3,11	3,99 ± 3,05
30×20	**2,95 ± 1,87**	2,97 ± 1,98	3,43 ± 1,90	5,89 ± 2,99	5,30 ± 2,89
50×40	2,01 ± 1,00	1,96 ± 1,04	**1,86 ± 1,03**	2,46 ± 0,92	—
100×80	1,57 ± 0,79	1,52 ± 0,60	**1,30 ± 0,71**	1,23 ± 0,53	—

Gap (%) respecto del experto ILS; menor es mejor. En negrita, el mejor de los tres modelos en cada tamaño. Como referencia, asignar las armas al azar da gaps de 259 a 588 %. La Figura 6, al final de la sección, muestra los mismos datos en barras.

- **En el tamaño de entrenamiento (10×8),** los tres modelos quedan entre 2,21 % y 2,44 %, muy por debajo de MMR (7,68 %) y mejor que el modelo del módulo 2 (2,97 %).
- **En tamaños que no vieron al entrenar,** siguen funcionando bien: hasta 30×20 se mantienen entre 0,7 % y 3,4 % (MMR: 5,7–7,8 %) y mejoran al módulo 2 en todos los tamaños comparables.
- **En las instancias más grandes,** MMR se vuelve muy bueno: en 50×40 los modelos todavía le ganan, pero en 100×80 MMR (1,23 %) queda levemente mejor en promedio. Ahí el propio experto ya no es óptimo: una búsqueda 20 veces más larga lo mejora en 0,8–1,1 %.
- **Entre las tres arquitecturas** las diferencias son pequeñas (a lo más 0,6 puntos) y del mismo tamaño que las diferencias entre semillas, así que ninguna gana claramente. Lo que distingue a B es que encuentra la misma solución que el experto con más frecuencia: en el 42 % de las ejecuciones de 10×8, contra 31–32 % de las otras dos.
- **En tiempo,** en 10×8 cada modelo resuelve una instancia en menos de 1 ms, entre 11 y 28 veces más rápido que el experto. En 100×80, en cambio, son más lentos que él, porque el modelo toma W decisiones y cada una revisa todos los pares.

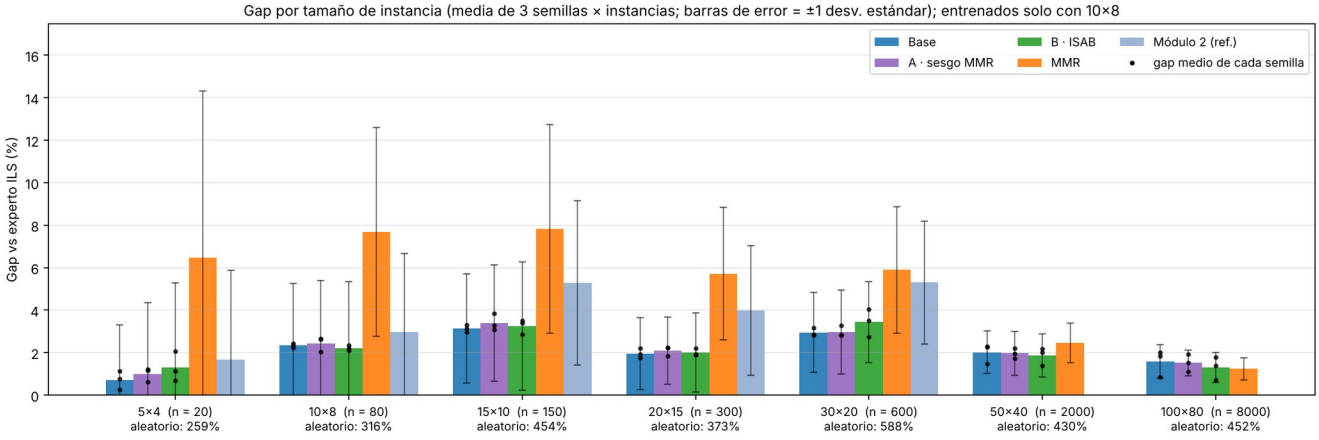


Figura 6. Gap por tamaño: la barra es el promedio, la línea vertical ±1 desviación estándar y los puntos negros, el promedio de cada semilla.

7. Conclusiones

- **El rediseño funciona.** Separar un encoder que lee la instancia una vez de un decoder que decide paso a paso dio un modelo más preciso que el del módulo 2 en todos los tamaños comparables y 36 veces más rápido en 50×40.

- **El costo es lineal.** Cada decisión crece en proporción al número de pares, que es lo mínimo posible; la variante B mantiene esa propiedad gracias a ISAB.
- **La pista de MMR (A) ayuda al comienzo,** porque el modelo parte desde una buena política, pero no mejora el resultado final: la base aprende lo mismo en una época.
- **La comunicación entre pares (B)** logra más soluciones iguales a las del experto, pero no un gap promedio menor, y entrenarla cuesta 4 veces más. Con estos resultados, la base es la opción con mejor relación entre calidad y costo.

Referencias

Ahuja, R. K., Kumar, A., Jha, K. C. y Orlin, J. B. (2007). Exact and heuristic algorithms for the weapon-target assignment problem. Operations Research, 55(6), 1136–1146.

den Broeder, G. G., Ellison, R. E. y Emerling, L. (1959). On optimum target assignments. Operations Research, 7(3), 322–326.

Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S. y Teh, Y. W. (2019). Set Transformer: a framework for attention-based permutation-invariant neural networks. ICML.

Ma, X., Kong, X., Wang, S., Zhou, C., May, J., Ma, H. y Zettlemoyer, L. (2021). Luna: Linear unified nested attention. NeurIPS.

Material del curso (TSP-Framework): notebooks de sesgo inductivo, atención lineal (LUNA, ISAB) y esparsificación.